

Bericht

Jiaqi Wang

17. Juli 2026

1 Ausführen der Host-Software unter Ubuntu

Die Host-Software wurde ursprünglich auf einem Windows-Computer mit Qt 5 entwickelt. Da die erzeugte Anwendung eine Windows-`.exe`-Datei ist, kann sie unter Ubuntu 22.04 nicht direkt nativ ausgeführt werden.

Zunächst wurde versucht, Qt 5 direkt unter Ubuntu 22.04 zu installieren und die Host-Software auf dem Linux-System erneut zu kompilieren. Dafür wurden die folgenden Befehle verwendet:

```
sudo apt-get install qt5-default
sudo apt update
sudo apt install build-essential qtbase5-dev qtbase5-dev-tools qt5-qmake
```

AnschlieSSend wurde überprüft, ob `qmake` korrekt installiert wurde:

```
qmake -v
```

Danach wurde versucht, den Quellcode der Host-Software mit `cmake` neu zu kompilieren:

```
cd '/home/itmhiwi/jwang/itm-hiwi-main/host_pc/test'
mkdir build
cd build
cmake ..
```

Dieser Ansatz ist jedoch aufwendiger, da die unter Windows entwickelte Qt-5-Anwendung möglicherweise zusätzliche Anpassungen für die Linux-Umgebung benötigt. AuSSerdem müssen Abhängigkeiten, Projektkonfigurationen und plattformspezifische Einstellungen überprüft und gegebenenfalls geändert werden.

Daher wurde anschlieSSend eine einfachere und effizientere Methode gewählt: die Ausführung der vorhandenen Windows-`.exe`-Datei unter Ubuntu mithilfe von Wine. Wine ist eine Kompatibilitätsschicht, mit der viele Windows-Programme unter Linux ausgeführt werden können, ohne dass eine vollständige Windows-Installation oder eine virtuelle Maschine benötigt wird.

1.1 Installation von Wine

Zunächst wird die 32-Bit-Architektur aktiviert, da viele Windows-Programme zusätzlich 32-Bit-Bibliotheken benötigen:

```
sudo dpkg --add-architecture i386
sudo apt update
```

Anschließend werden Wine und die benötigten Zusatzpakete installiert:

```
sudo apt install wine wine64 wine32 winetricks
```

Die installierte Wine-Version kann mit folgendem Befehl überprüft werden:

```
wine --version
```

1.2 Starten der Host-Software mit Wine

Danach wird in das Verzeichnis gewechselt, in dem sich die Windows-`.exe`-Datei befindet:

```
cd '/home/itmhiwi/jwang/MotroController_APP_V1'
cd pkg
```

Die Host-Software kann anschließend mit Wine gestartet werden:

```
wine MotroController.exe
```

2 Aktualisierung der PC-Software

Im Rahmen der Weiterentwicklung wurde die Benutzeroberfläche der PC-Software überarbeitet. Ziel der Anpassungen war es, die Bedienung übersichtlicher zu gestalten und die Einstellung des Motors intuitiver zu machen. Die wichtigsten Änderungen sind im Folgenden zusammengefasst:

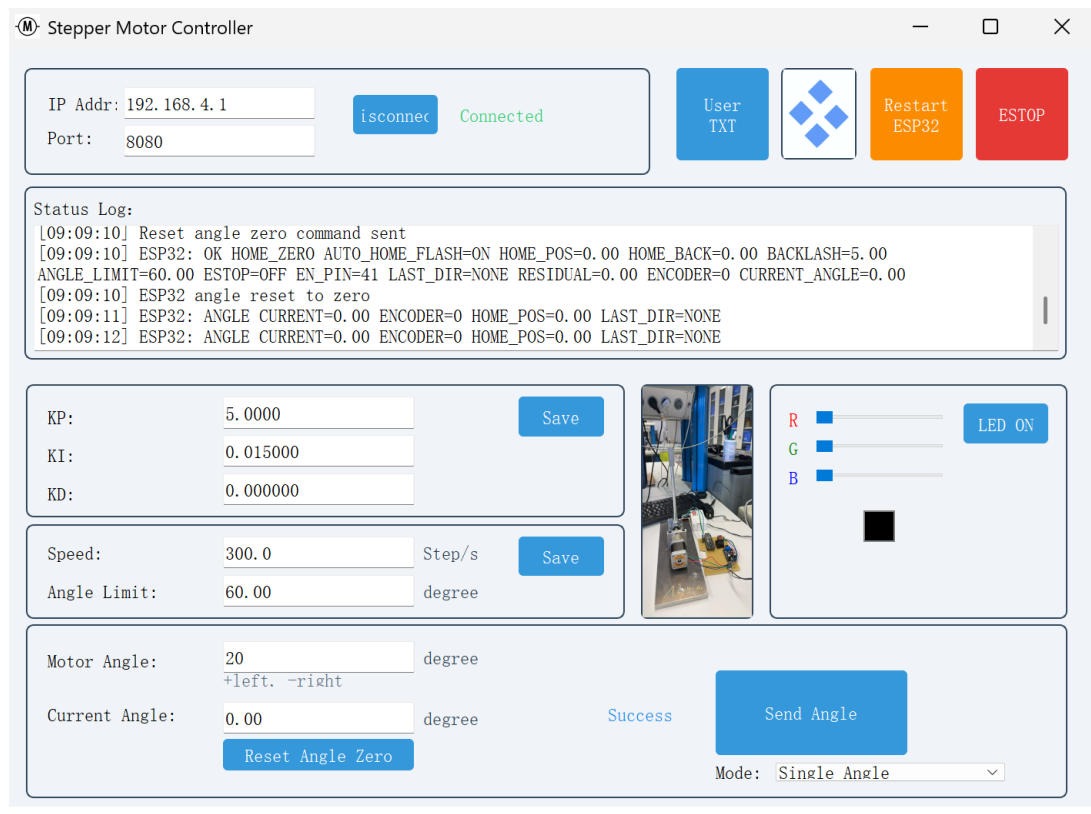


Abbildung 1: Aktualisierte Benutzeroberfläche der PC-Software

- **Anpassung der Logo-Position:**

Das Logo wurde in die obere rechte Ecke der Benutzeroberfläche verschoben. Dadurch nimmt es weniger Platz im zentralen Bedienbereich ein und beeinträchtigt die Darstellung der Steuerungselemente nicht.

- **Trennung der PID-Parameter von den Betriebsparametern:**

Die PID-Reglerparameter K_p , K_i und K_d wurden von den Parametern *Speed* und *Angle Limit* räumlich getrennt. Da die PID-Parameter nach der Inbetriebnahme nur selten verändert werden, sind sie nun in einem separaten Einstellungsbereich angeordnet. Die häufiger verwendeten Betriebsparameter bleiben dadurch leichter zugänglich.

- **Ergänzung der Winkeleinheit:**

Bei allen winkelbezogenen Eingabefeldern und Anzeigen wurde die Einheit Grad ($^{\circ}$) ergänzt. Dadurch ist die Bedeutung der eingegebenen und angezeigten Werte eindeutig erkennbar.

- **Darstellung des Motors in der Vorderansicht:**

In der Mitte der Benutzeroberfläche wurde eine Vorderansicht des Motors eingefügt. Diese Darstellung ermöglicht eine intuitive Zuordnung der Drehrichtung: Ein positiver Winkel entspricht einer Drehung nach links, während ein negativer Winkel eine Drehung nach rechts beschreibt.

Nach Abschluss der Entwicklung wurde die Qt-Anwendung zusammen mit allen erforderlichen Laufzeitbibliotheken und Ressourcendateien in einem separaten Ordner bereitgestellt. Als Entwicklungsumgebung wurde Qt 5.15.19 mit dem MinGW-64-Bit-Compiler verwendet.

Zunächst wird im Verzeichnis

```
host_pc/test/build
```

ein neuer Ordner mit dem Namen `pkg` erstellt. AnschlieSSend werden die ausführbare Datei und die benötigten Ressourcendateien aus dem Build-Verzeichnis

```
Deskt_QT_5_15_19_MinGW_64_bit-Debug
```

in den Ordner `pkg` kopiert.

Die folgenden Befehle werden in einer Windows-Eingabeaufforderung ausgeführt:

```
cd host_pc\test\build

mkdir pkg

copy Deskt_QT_5_15_19_MinGW_64_bit-Debug\test.exe pkg\
copy Deskt_QT_5_15_19_MinGW_64_bit-Debug\top_middle_image.png pkg\
copy Deskt_QT_5_15_19_MinGW_64_bit-Debug\middle_image.png pkg\
copy Deskt_QT_5_15_19_MinGW_64_bit-Debug\user.txt pkg\
```

Listing 1: Vorbereitung des Paketordners

Nach dem Kopieren kann der Name der ausführbaren Datei geändert werden. In diesem Fall wird die Datei `test.exe` in `MotorController.exe` umbenannt:

```
cd pkg
ren test.exe MotorController.exe
```

Listing 2: Umbenennung der ausführbaren Datei

Die Umbenennung ändert lediglich den angezeigten Dateinamen und hat keinen Einfluss auf die grundlegende Funktion des Programms.

Danach wird die zur verwendeten Qt- und MinGW-Version passende Qt-Kommandozeile geöffnet. Dabei muss darauf geachtet werden, dass die Qt-Kommandozeile für Qt 5.15.19 und MinGW 64-Bit verwendet wird.

AnschlieSSend wird in den zuvor erstellten Paketordner gewechselt:

```
cd host_pc\test\build\pkg
```

Listing 3: Wechsel in den Paketordner

Falls sich das Projekt auf einem anderen Laufwerk befindet, kann in der Windows-Eingabeaufforderung zusätzlich die Option `/d` verwendet werden:

```
cd /d C:\Pfad\zu\host_pc\test\build\pkg
```

Listing 4: Wechsel des Laufwerks und Verzeichnisses

Mit dem Werkzeug `windeployqt` werden anschließend alle für die Ausführung erforderlichen Qt-Bibliotheken, Plugins und Compiler-Laufzeitdateien automatisch in den Paketordner kopiert. Da die ausführbare Datei zuvor umbenannt wurde, muss beim Aufruf ebenfalls der neue Dateiname verwendet werden:

```
windeployqt --debug --compiler-runtime MotorController.exe
```

Listing 5: Bereitstellung der Qt-Laufzeitbibliotheken

Die Option `-debug` wird verwendet, da die Anwendung aus dem Debug-Build-Verzeichnis stammt. Die Option `-compiler-runtime` stellt zusätzlich die erforderlichen MinGW-Laufzeitbibliotheken bereit.

Nach erfolgreicher Ausführung enthält der Ordner `pkg` neben der Datei `MotorController.exe` unter anderem die benötigten Qt-DLL-Dateien, Plattform-Plugins, Compiler-Laufzeitbibliotheken sowie die folgenden Ressourcendateien:

- `top_middle_image.png`
- `middle_image.png`
- `user.txt`

Abschließend wird der vollständige Paketordner als ZIP-Datei komprimiert.

Die erzeugte Datei `MotorController_APP_V1.1.zip` enthält das vollständige Programmpaket. Sie kann auf einen anderen Windows-PC übertragen und dort entpackt werden.

Nach dem Entpacken kann das Programm direkt über die Datei `MotorController.exe` gestartet werden, ohne dass auf dem Zielcomputer eine separate Qt-Installation erforderlich ist.

3 Lötarbeiten

Das ESP32-S3-Modul, der 12V-zu-5V DC/DC-Wandler und das A4988-Schrittmotortreibermodul wurden auf einer Lochrasterplatine montiert und verlötet.

Zusätzlich wurden eine Schraubklemme für den 12-V-Eingang sowie ein Anschluss für den EC11-Drehgeber herausgeführt. Für einen späteren optischen Sensor wurde auSSerdem ein freier Anschluss vorgesehen.

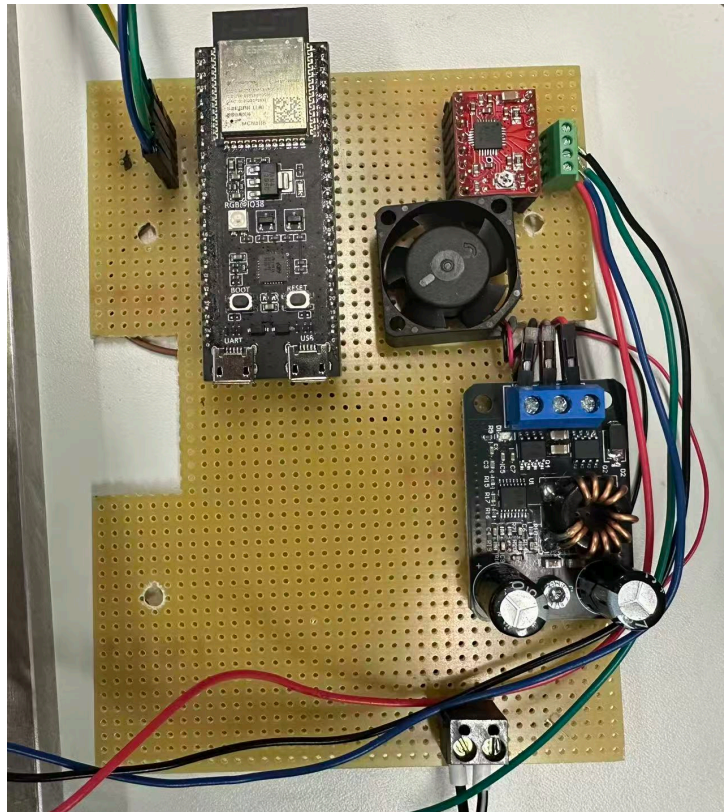


Abbildung 2: Gelötete Steuerplatine auf der Lochrasterplatine

4 Messung der Leistungsaufnahme

Für den Test wurde die Schaltung über ein Labornetzgerät mit 12 V versorgt. Dabei zeigte sich, dass das ESP32-S3-Modul und der EC11-Drehgeber nur einen geringen Anteil der Gesamtleistung aufnehmen. Der gröSSte Anteil entfällt auf das A4988-Modul und den Schrittmotor.

Der Motorstrom wurde über das Potentiometer des A4988-Moduls eingestellt. Bei etwa 400 mA arbeitete der Motor zuverlässig und mit zufriedenstellender Leistung. Eine Einstellung bis ungefähr 1,6 A ist möglich, führt jedoch zu einer sehr starken Erwärmung des A4988-Moduls. Bei einem Strom von 100 mA konnte sich der Motor nicht drehen.

Aus diesem Grund wurde ein Motorstrom von etwa 400 mA als geeigneter Betriebswert gewählt.

5 To-Do-List

Für die nächsten Versuche ist eine Versorgung über eine 12-V-Batterie vorgesehen. Da der Versuchsaufbau in der Mitte des Labors platziert werden soll, wären bei der Verwendung eines Labornetzgeräts möglicherweise sehr lange Stromleitungen erforderlich. Aus diesem Grund ist eine direkte Batterieversorgung praktischer.

Für den Anschluss der Batterie müssen geeignete Leitungen und Steckverbinder ausgewählt und angepasst werden. Dabei sind insbesondere der erforderliche Strom, die Polarität und eine sichere Befestigung zu berücksichtigen.

Zusätzlich soll mit der Software Inkscape eine Grundplatte entworfen werden. Auf dieser Grundplatte sollen der Motor, die Steuerplatine und die Batterie gemeinsam befestigt werden, sodass ein kompakter und stabiler Versuchsaufbau entsteht.

Ein weiterer Arbeitsschritt ist die Ansteuerung des ESP32-S3 in einer ROS-2-Umgebung. Dadurch soll die Steuerung später in den Versuchsaufbau integriert und für weitere Experimente vorbereitet werden.

- 12-V-Batterie als Spannungsversorgung verwenden,
- geeignete Leitungen und Anschlüsse für die Batterie vorbereiten,
- Grundplatte mit Inkscape entwerfen,
- Motor, Steuerplatine und Batterie auf der Grundplatte befestigen,
- Kommunikation und Steuerung des ESP32-S3 unter ROS 2 realisieren.